MIT 8103 Advanced Database Systems: Portfolio Summary

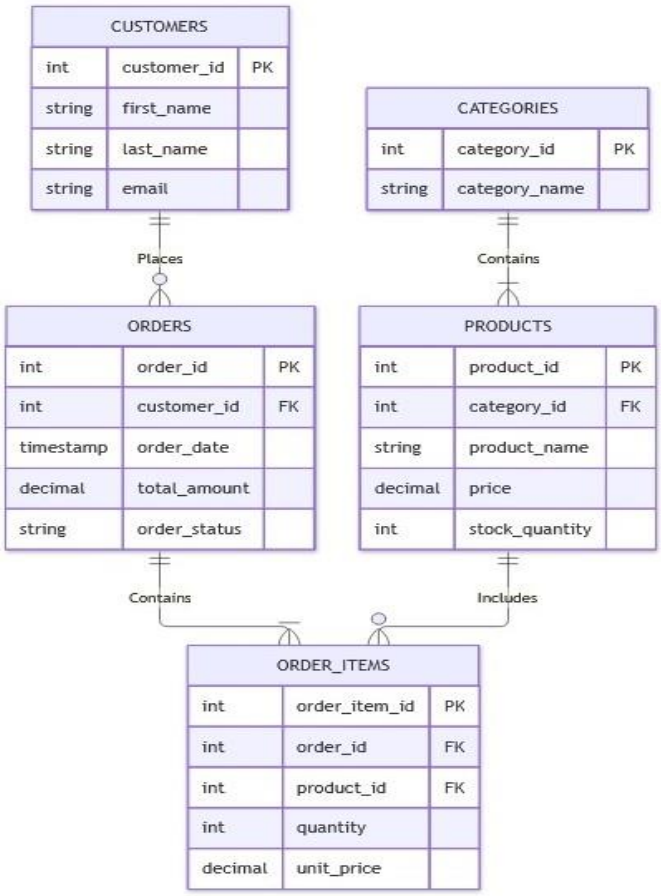
Name: Korede Akintunde
Student ID: 301915156
Case study: E-commerce Platform
Repository: https://github.com/kakintunde5156-prog/Akintunde_Korede_301915156

Case study

The RanGo e-commerce platform manages customers, products, orders, and items. It requires strict relational data integrity for financial transactions (via PostgreSQL) and a flexible NoSQL architecture for event logging (via MongoDB), both running within a single GitHub Codespace for this demonstration.

Portfolio 1: Database Design and Modelling

The relational schema uses four tables: customer, product, order, and order_item. Separating orders from individual items allows multiple products per checkout without duplicating order totals. Primary and foreign keys ensure referential integrity. CHECK constraints are applied so total_amount and quantity cannot be negative numbers, protecting financial logic. The design is normalized to 3NF, eliminating repeating groups and data duplication. Six invalid inserts were successfully rejected during testing.



Portfolio 2: Query Processing and Optimization

To test performance, 100,000 synthetic order records were generated. A query filtering by order_date initially required a full Sequential Scan. After applying a B-Tree index to order_date, the execution plan improved to an Index Only Scan. Measured median times (after a warm-up run) were:

Measurement	Before index	After index
Scan Type	Seq Scan	Index Only Scan
Median execution time	8.7 ms	0.04 ms
Shared buffer hits, third run	834	3

The index drastically reduced execution time by jumping directly to matching dates, avoiding unrelated rows. However, these shared Codespace measurements exclude the overhead cost of maintaining the index during heavy insert operations.

Portfolio 3: Transactions and Concurrency

This section demonstrated ACID properties. Using COMMIT, a valid update was permanently saved. A ROLLBACK successfully discarded a transaction block containing a constraint violation, keeping the database in a consistent state. To test concurrency, two READ COMMITTED sessions attempted to update the same order. Session B was blocked by Session A's uncommitted row-level lock. When Session A finally committed, Session B re-evaluated the WHERE status = 'pending' condition, found it unmet, and updated zero rows, safely preventing a lost update.

Portfolio 4: NoSQL and Advanced Data Models

A MongoDB document model was added for flexible e-commerce event logging. Four sample documents in the rango_events database embed customer and product details, creating self-contained snapshots. Utilizing flexible schemas, "order_placed" events store a payment method, while "order_shipped" events store a carrier and tracking number. The database acknowledged inserts E001–E004 and successfully executed nested queries. These are isolated NoSQL examples and do not automatically sync from PostgreSQL.

Portfolio 5: Distributed and Cloud Database Exercise

To protect MongoDB event records, a 3-node replica set (one Primary, two Secondaries) across separate failure zones is proposed. If the Primary fails, Secondaries elect a replacement. To prevent data loss, the system requires majority write acknowledgement (w: majority). The current low event volume does not justify sharding. A managed cloud deployment (like Atlas) provides automated backups and encryption, but MongoDB replication does not automatically protect or sync the separate PostgreSQL database.

Evidence

All SQL/JS scripts, setup documentation, and execution screenshots are maintained in the GitHub repository under their respective folders.

Individual reflection

Design choice: I separated the order and order_item tables and applied strict CHECK constraints. This allows multiple products per order without duplicating the total price, while ensuring quantities and totals never drop below zero to prevent checkout bugs.

Difficulty, Action, and Lesson: In Portfolio 3, Session B's terminal appeared frozen during a concurrent update. I realized it was intentionally blocked because Session A held an uncommitted row-level lock. Running COMMIT; in Session A released the lock, allowing Session B to finish. This taught me that basic availability checks are insufficient in multi-user systems; row-level locks are essential to prevent users from overwriting each other.

References

- PostgreSQL Global Development Group. PostgreSQL 16: Constraints. https://www.postgresql.org/docs/16/ddl-constraints.html
- PostgreSQL Global Development Group. PostgreSQL 16: Using EXPLAIN. https://www.postgresql.org/docs/16/using-explain.html
- PostgreSQL Global Development Group. PostgreSQL 16: Transaction Isolation. https://www.postgresql.org/docs/16/transaction-iso.html
- MongoDB. Data Modeling in MongoDB. https://www.mongodb.com/docs/manual/data-modeling/
- MongoDB. Replication. https://www.mongodb.com/docs/manual/replication/
- MongoDB. Replica Set Elections. https://www.mongodb.com/docs/v8.0/core/replica-set-elections/
- MongoDB. Write Concern. https://www.mongodb.com/docs/v8.0/reference/write-concern/

AI-use declaration

I used Gemini pro to help structure my markdown files and explain Codespaces error messages. I executed the database commands myself, checked the results against the assessment requirements, and verified all outputs locally before submission. I understand the work and remain responsible for its accuracy.